

Assignment 5: Research Implementation

Abstract

This project presents a real-time cel-shading system implemented in OpenGL, inspired by the paper "Engine-Integrated Cel-Shading for Unreal Engine 5" [1]. The aim was to recreate stylised non-photorealistic rendering (NPR) techniques without relying on a game engine, instead building a custom shader-based pipeline. The system implements key cel-shading features, including quantised lighting, rim lighting, stylised specular highlights, and outline rendering. Furthermore, an interactive control system allows real-time adjustment of parameters such as band thresholds, outline thickness, and lighting behaviour. A preset system was also developed to replicate different stylised looks inspired by well-known cel-shaded games. Multiple 3D models with textures are supported through Assimp and stb_image, allowing both embedded and external textures to be used. The end result is an interactive rendering system capable of producing a variety of stylised visual outputs while maintaining real-time performance.

Background

The chosen paper, "Engine-Integrated Cel-Shading for Unreal Engine 5", describes how to incorporate cel-shading techniques directly into a modern game engine pipeline. The work shows how to integrate stylised rendering into an existing physically based rendering (PBR) framework while maintaining performance and flexibility. The main premise behind cel-shading is to reduce lighting to discrete bands rather than continuous gradients. Resulting in a flat, cartoon-like appearance, as seen in animation and stylised video games. The paper discusses how to implement such techniques in a way that works with engine-level systems like lighting, materials, and postprocessing.

Several key techniques are highlighted: Lighting quantisation (banded shading), stylised shadow control, integration with real-time rendering pipelines, and artistic control of shading parameters.

These concepts were adapted and put into action in this project's standalone OpenGL environment. Rather than integrating into an engine, the emphasis was on developing a shader-driven system that replicated similar visual effects. Core techniques like quantised lighting, rim lighting, and outline rendering were implemented, but advanced engine-level features like full post-processing pipelines were not.

Implementation Details

The implementation of the proposed cel-shading system focuses on translating the core ideas presented in the selected paper into a standalone OpenGL-based rendering pipeline. Rather than relying on an engine-level framework, all functionality was developed manually using GLSL shaders and standard graphics programming techniques. The goal was to reproduce key stylised rendering effects while maintaining interactivity and flexibility. This section outlines the structure of the system, the main rendering techniques used, and how user interaction was incorporated to facilitate real-time control over the visual output.

System Overview

The system is implemented using OpenGL 3.3 and GLSL shaders. It consists of a real-time rendering pipeline capable of loading 3D models, applying stylised shading, and rendering them interactively. The system's main components include mesh and texture loading, a cel-shading fragment shader, an outline rendering pass, an interactive control system, and preset-based stylisation. Vertex and fragment shaders are used for rendering, and data is passed through vertex buffers. The system supports a variety of models and textures, allowing the user to switch between them during runtime.

Mesh and Texture Loading

3D models are imported using the Assimp library, supporting both .dae and .obj formats. During loading, vertex positions, normals, and texture coordinates are extracted and stored in buffers for rendering.

```
simpleVertexShader.txt
vec4 worldPos = model * vec4(vertex_position, 1.0);
FragPos = worldPos.xyz;
Normal = mat3(transpose(inverse(model))) * vertex_normal;
TexCoords = vertex_uv;

gl_Position = proj * view * worldPos;
```

For .obj models, material information is also parsed. If a diffuse texture is defined in the associated .mtl file, the texture is loaded automatically using stb_image. This allows models to retain their original textures without requiring manual reassignment. If a model does not include an embedded texture, a fallback system allows the user to select from a set of predefined textures.

```
main.cpp
if (mesh->mMaterialIndex >= 0 && !modelData.hasEmbeddedTexture) {
    aiMaterial* material = scene->mMaterials[mesh->mMaterialIndex];
    aiString texPath;

    if (material->GetTexture(aiTextureType_DIFFUSE, 0, &texPath) == AI_SUCCESS) {
        std::string fullPath = directory + texPath.C_Str();
        modelData.diffuseTex = loadTextureFromFile(fullPath.c_str());
        modelData.hasEmbeddedTexture = true;
    }
}
```

This allows models to automatically use their embedded material textures when available, reducing manual setup and improving visual consistency across imported assets

Cel-Shading Shader

The system is built around a fragment shader that uses lighting quantisation to implement cel-shading. First, the surface normal and light direction are used to calculate a standard Lambertian diffuse lighting model. This continuous lighting value is then divided into bands with configurable threshold values.

```
simpleFragmentShader.txt
float getSteppedBand(float intensity){
    int thresholdsToUse = bandCount - 1;
    int bandIndex = 0;
    for (int i = 0; i < thresholdsToUse; i++) {
        if (intensity > bandThresholds[i]) {
            bandIndex++;
        }
    }
    return float(bandIndex) / float(max(bandCount - 1, 1));
}
```

Lighting is first computed using Lambertian reflectance, then quantised into discrete bands using configurable thresholds, forming the basis of the cel-shading effect

The number of bands and their thresholds can be adjusted at runtime, allowing different levels of stylisation. Lower band counts create a more cartoon-like appearance, whereas higher band counts produce a slightly smoother result. Shadow and highlight tinting provide additional colour control.

```
simpleFragmentShader.txt
vec3 getBandTint(float stepped){
    if (stepped <= 0.33) return shadowBandColor;
    if (stepped <= 0.66) return midBandColor;
    return lightBandColor;
}
(...)
vec3 bandTint = getBandTint(stepped);
vec3 color = baseColor * bandTint;
color = mix(color * shadowTint, color * highlightTint, stepped);
```

This allows different artistic styles to be achieved by adjusting the colour of the dark and light regions separately.

Stylisation Features

Several additional effects were implemented to enhance the cel-shaded appearance:

➤ **Specular Highlights**

Specular reflections are implemented using a thresholded approach, producing sharp, stylised highlights instead of smooth reflections.

```
simpleFragmentShader.txt
float spec = pow(max(dot(V, R), 0.0), 32.0);
float toonSpec = spec > specularThreshold ? 1.0 : 0.0;
color += vec3(0.95) * toonSpec;
```

➤ Rim Lighting

Rim lighting is calculated based on the view direction relative to the surface normal. This enhances object silhouettes and adds visual depth.

```
simpleFragmentShader.txt
float rim = 1.0 - max(dot(V, N), 0.0);
float toonRim = rim > rimThreshold ? rimAmount : 0.0;
color += vec3(0.25, 0.25, 0.35) * toonRim;
```

➤ Outline Rendering

Outlines are rendered using a second pass. The model is drawn again with front-face culling enabled and slightly expanded geometry, producing a visible silhouette around the object.

```
outlineVertexShader.txt
vec3 expandedPos = vertex_position + normalize(vertex_normal) * outlineWidth;
vec4 worldPos = model * vec4(expandedPos, 1.0);
gl_Position = proj * view * worldPos;
```

```
main.cpp
if (useOutline) {
    glEnable(GL_CULL_FACE);
    glCullFace(GL_FRONT);
    (...)
```

Front-face culling ensures only the expanded back-faces are rendered, creating the visible outline.

➤ Band Colour Control

Separate colours are used for shadow, mid-tone, and light regions, allowing fine control over the final appearance. (see *Cel-Shading Shader*)

Interactivity and Presets

A real-time control system via keyboard input was implemented. This allows users to modify rendering parameters dynamically, including band count and thresholds, outline thickness, specular and rim lighting toggles, texture and mesh selection, and light movement behaviour. A preset system was also developed to store predefined configurations.

```

main.cpp
void applyPreset(int index){
    if (index < 0 || index >= (int)presets.size()) return;
    currentPreset = index;
    const CelPreset& p = presets[index];
    bandCount = p.bandCount;
    for (int i = 0; i < 5; i++){
        bandThresholds[i] = p.bandThresholds[i];
    }
    specularThreshold = p.specularThreshold;
    rimThreshold = p.rimThreshold;
    rimAmount = p.rimAmount;
    outlineWidth = p.outlineWidth;

    useTexture = p.useTexture;
    useSpecular = p.useSpecular;
    useRim = p.useRim;
    useOutline = p.useOutline;

    shadowTint = p.shadowTint;
    highlightTint = p.highlightTint;

    shadowBandColor = p.shadowBandColor;
    midBandColor = p.midBandColor;
    lightBandColor = p.lightBandColor;
}

```

```

main.cpp
void setupPresets()
{
    presets.clear();

    presets.push_back({
        "Custom",
        5, // bandCount
        {0.15f, 0.35f, 0.55f, 0.75f, 0.90f}, // bandThresholds
        0.75f, // specularThreshold
        0.60f, // rimThreshold
        0.20f, // rimAmount
        0.025f, // outlineWidth
        false, true, true, true, // useTexture, useSpecular, useRim, useOutline
        glm::vec3(0.55f, 0.70f, 0.85f), // shadowTint
        glm::vec3(1.00f, 0.96f, 0.82f), // highlightTint
        glm::vec3(0.28f, 0.40f, 0.70f), // shadowBandColor
        glm::vec3(0.72f, 0.78f, 0.72f), // midBandColor
        glm::vec3(1.00f, 0.93f, 0.70f) // lightBandColor
    });
}
(...)

```

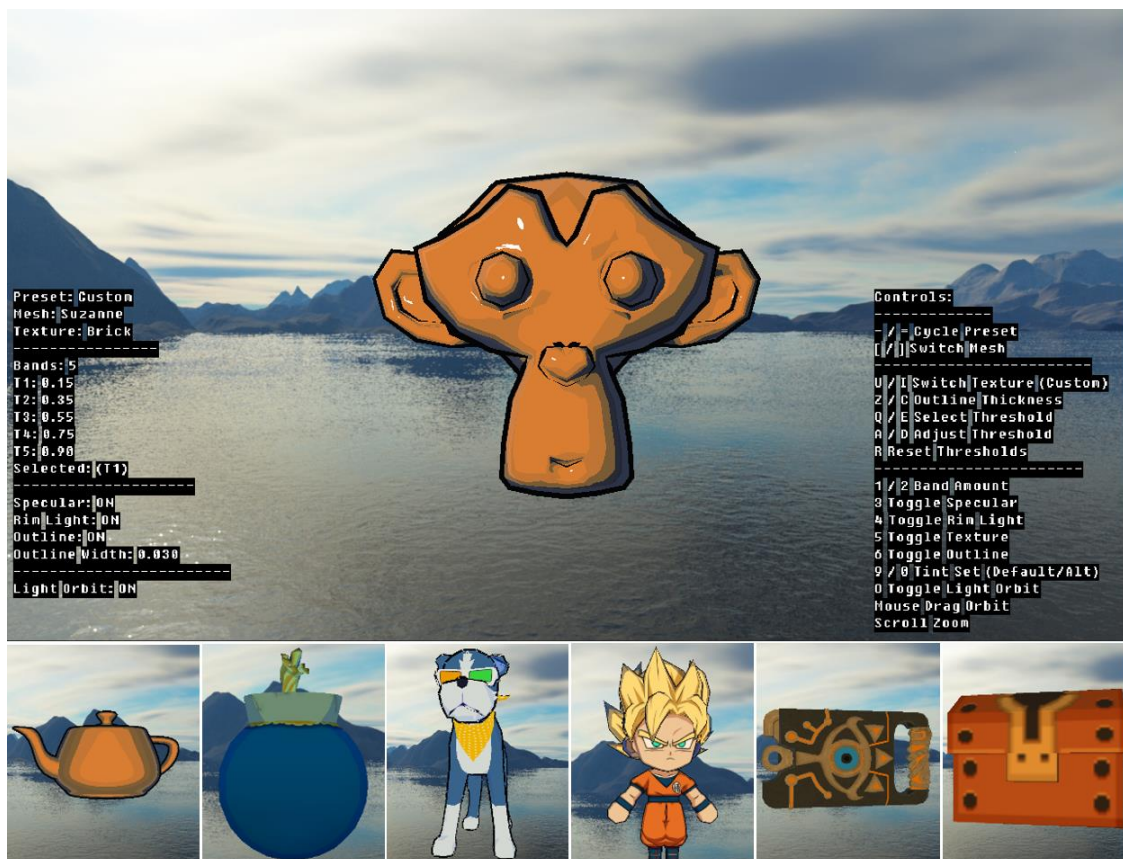
The created presets are inspired by different cel-shaded styles and allow rapid switching between visual looks during demonstration, improving usability compared to manually adjusting parameters.

Camera and Lighting

The camera system allows the user to orbit around the model using mouse input, with adjustable zoom. This enables the scene to be viewed from different angles. Lighting can operate as either a static light source or an orbiting light source. The orbiting light mode rotates the light around the object, allowing the shading behaviour to be observed at runtime. A visual marker is also rendered to indicate the light position.

```
main.cpp
if (orbitLight) {
    float lightAngle = glm::radians(rotate_y);
    currentLightPos = glm::vec3(
        10.0f * cos(lightAngle),
        2.5f,
        10.0f * sin(lightAngle)
    );
}
else {
    currentLightPos = glm::vec3(0.0f, 0.0f, 15.0f);
}
```

Results and Evaluation



The system achieves real-time performance while rendering multiple models using stylised shading effects. The computational cost is kept low by using relatively simple lighting calculations and minimal post-processing. The cel-shading implementation yields clear and distinct results. Banded lighting, combined with outline rendering and rim lighting, produces a strong, stylised appearance.

The ability to adjust parameters in real time allows for the exploration of a variety of visual styles. The preset system significantly increases the demo's usability. It enables different styles to be demonstrated quickly without requiring manual adjustments to multiple parameters. This is especially useful for displaying variations influenced by various games. The system efficiently handles multiple textured models, including those with embedded material textures. Performance remains stable even when switching between models and presets during runtime.

Limitations and Improvements

While the system successfully implements core cel-shading techniques, it remains a simplified approximation of engine-integrated solutions. One limitation is the lack of advanced post-processing effects. Screen-space edge detection, halftoning, and hatching could all help to improve the stylised appearance. Additionally, shadow mapping is not supported, limiting realism in more complex scenes.

In comparison to modern game engines, the implemented lighting model is minimal. While adequate for stylised rendering, it does not fully replicate the complexities of engine-based shading systems. Some presets are merely approximations of their intended styles. Closer matches would necessitate additional effects and more precise artistic tuning.

Future improvements that could elevate the quality of this rendering implementation include:

➤ ***Screen-space outline detection:***

The current implementation generates outlines using a geometry-based approach, where the mesh is rendered a second time with inverted face culling and slight scaling. While effective, this method can produce inconsistent outline thickness depending on camera distance and model complexity. A screen-space edge detection technique, based on depth or normal discontinuities, would allow for more consistent and visually accurate outlines. This approach would also better match modern cel-shading pipelines used in game engines.

➤ ***Additional NPR effects (e.g., halftone shading):***

The current system focuses primarily on banded lighting and basic stylisation techniques. Incorporating additional non-photorealistic rendering effects such as halftone shading, cross-hatching, or gradient mapping would significantly enhance the artistic range of the system. These techniques could be implemented in fragment shaders or as post-processing passes, allowing for more expressive and varied visual styles beyond traditional cel-shading.

➤ **Support for animated models:**

At present, the system only supports static meshes. Adding support for skeletal animation would allow animated characters to be rendered with cel-shading, making the system more applicable to real-world use cases such as games or animated scenes. This would require integrating animation data from Assimp and modifying the vertex shader to support skinning calculations.

➤ **More complex scene composition and lighting setups:**

The current implementation renders a single object with a single light source. Expanding the system to support multiple objects, multiple lights, and more complex scene arrangements would improve realism and flexibility. Additionally, implementing shadow mapping would allow objects to cast shadows onto each other, further enhancing depth and visual quality while maintaining the stylised aesthetic.

References

- [1]. G. P. L. de Lucena, C. Madeira, C. Rennó-Costa, and A. Souza, *Engine-Integrated Cel-Shading for Unreal Engine 5*, Proceedings of the SIGGRAPH Asia 2025 Posters, Dec. 2025. DOI: 10.1145/3757374.3771456
- [2]. Blender Foundation, *Blender Monkey (Suzanne)*
- [3]. Utah Teapot, Sketchfab: <https://sketchfab.com/3d-models/the-utah-teapot-1092c2832df14099807f66c8b792374d>
- [4]. Spriters Resource (Various Models): <https://models.spriters-resource.com/>
- [5]. SS Goku [Lobby], Sketchfab: <https://sketchfab.com/3d-models/baby-goku-super-saiyan-6747a6e7a5834d68931132c80a356444>
- [6]. LearnOpenGL: <https://learnopengl.com/>
- [7]. OpenGL Reference (Khronos): <https://registry.khronos.org/OpenGL/>
- [8]. GLFW Documentation: <https://www.glfw.org/documentation.html>
- [9]. GLM Documentation: <https://glm.g-truc.net/>
- [10]. Assimp Documentation: <https://assimp-docs.readthedocs.io/>
- [11]. stb_image: <https://github.com/nothings/stb>